

FOC Auto-Tuning Guide

Power Application Controller™





Table of Contents

1 Introduction	3
2 Description	4
2.1 Components	4
2.1.1 Motor ID (MID)	4
2.1.2 PI Controller Auto-Tuning (ATPI)	4
2.2 Auto-Tuning Procedure Summary	5
3 Using Auto-Tuning	6
3.1 Enabling Auto-Tuning	6
3.2 GUI Controls and Displays	7
3.2.1 General Configuration	7
3.2.2 MID1	7
3.2.3 MID2	8
3.2.4 ATPI	8
3.3 Auto-Tuning Procedure	10
3.3.1 Initial Steps	10
3.3.2 Running Auto-Tuning	11
3.4 Warnings and Faults	13
3.5 Saving the Calculated Values	14
3.6 Firmware Parameters	15
3.7 Backward Compatibility	17
Contact Information	18
Important Notice	18



1 INTRODUCTION

Auto-Tuning is a feature that uses known motor and system parameters to automatically calculate appropriate values for the proportional and integral gains, K_p and K_i respectively, used in the PI controllers for current and speed. This feature accomplishes the following goals:

- 1) Dramatically reduces the amount of time that the user needs to spend tuning a new motor
- 2) Simplifies the tuning process for users who are not proficient in motor tuning
- 3) Avoids system instability that is possible when manually tuning the controllers

This feature is specifically useful in the following cases:

- 1) Large number of motors need to be tuned by the user
- 2) The user is not very experienced with motor tuning
- 3) The user needs to very quickly be able to tune a motor at a customer site

Auto-Tuning has the following limitations:

- 1) It can only be used for tuning current and speed PI controllers. It is currently unable to tune PI controllers in other modes of operation such as power or voltage control.
- 2) It can only be used to tune speed controllers for systems where the load inertia is constant over time and speed, such as fans. If the load inertia is dynamic, as in a pump or compressor, the speed controller tuning may not be accurate.
- 3) For motor and load characterization done by Auto-Tuning, the user must be able to spin the motor with the actual load that the motor will see in the application.



2 DESCRIPTION

2.1 Components

There are two main components to Auto-Tuning:

2.1.1 Motor ID (MID)

Identifies and calculates the necessary motor parameters and load characteristics for sensorless motor control and PI controller tuning. MID can be further subdivided into distinct sequences:

- a. MID1 is a sequence that injects a sequence of voltages to the motor windings while the motor is stationary in order to identify the following motor parameters:
 - i. Motor winding resistance (R_s)
 - ii. Motor winding inductance (L_d & L_q)
- b. MID2 is a sequence run while the motor is being driven and in motion in order to identify the following motor and load parameters:
 - i. Motor BEMF constant (K_e)
 - ii. Load inertia (J)

2.1.2 PI Controller Auto-Tuning (ATPI)

Calculates appropriate proportional and integral gains for the current and speed PI controllers by using known motor parameters and load characteristics obtained in MID1/2.

2.2 Auto-Tuning Procedure Summary

The general steps to follow during the Auto-Tuning process are listed below:

1) Run MID1 Sequence

Because Current ATPi requires motor winding inductance and resistance (L_d , L_q , and R_s), the MID1 sequence must be run first to obtain these values. Alternatively, if the user already has these values, MID1 can be skipped and the user can enter these values manually.

2) Calculate Current ATPi

Tuning the current PI controllers is purely a calculation based on known motor and system parameters and therefore requires no specific action by the user other than entering the necessary values into the GUI. The necessary values are L_d , L_q , R_s , and speed filter cutoff frequency.

3) Start motor in torque mode

If the Open-Loop startup method has not been tuned, the FOC Tuning Guide can be referenced to complete this step. The motor needs to be driven in torque mode in order to run the MID2 sequence.

4) Run MID2 Sequence

The MID2 sequence is used to obtain parameter values for the motor BEMF constant (K_e) and load inertia (J). These values are required for calculating the appropriate gain values for the Speed PI controller. Alternatively, if the user already has these values, MID2 can be skipped and the user can enter these values manually.

During the MID2 sequence, two main processes occur:

- a. A step increase in torque is applied to the motor and the speed increases as a result. The speed change that results from this torque step increase is monitored to calculate the inertia of the load.
- b. The drive voltage required to maintain a certain speed is processed to obtain the motor's BEMF constant.

Note that because the MID2 sequence requires running the motor with load in torque mode, having tuned current PI controllers is necessary before running MID2.

5) Calculate Speed ATPi

The speed PI controller gains are calculated as a function of the K_e and J that were just calculated as well other parameters such as speed filter cutoff frequency, desired speed response damping factor, and the number of pole pairs.



3 USING AUTO-TUNING

3.1 Enabling Auto-Tuning

As described earlier, Auto-Tuning consists of two components which must be enabled in the config_features.h file when the project is compiled:

ENABLE_AUTOTUNE_MID

This component is necessary to calculate motor parameters (Ld, Lq, R, Ke, and J). If the user already has these parameters it is not required to enable this component.

ENABLE_AUTOTUNE_PI

This component calculates the PI controller gains based on the motor parameters that are either obtained from the MID component or entered by the user.

When Auto-Tuning is compiled with the project, it is resident in memory and available to the user when necessary. Even when it is resident in memory, it does not interfere with normal operation of the motor.

3.2 GUI Controls and Displays

The Auto-Tuning tab is found in the Tuning panel of the GUI. Note that this tab is visible only if ATPi is enabled in the firmware. The Auto-Tuning tab is shown in Figure 1.

The ATPi tab is divided into 4 sections:

- 1) General Configuration.
- 2) MID1
- 3) MID2
- 4) ATPi

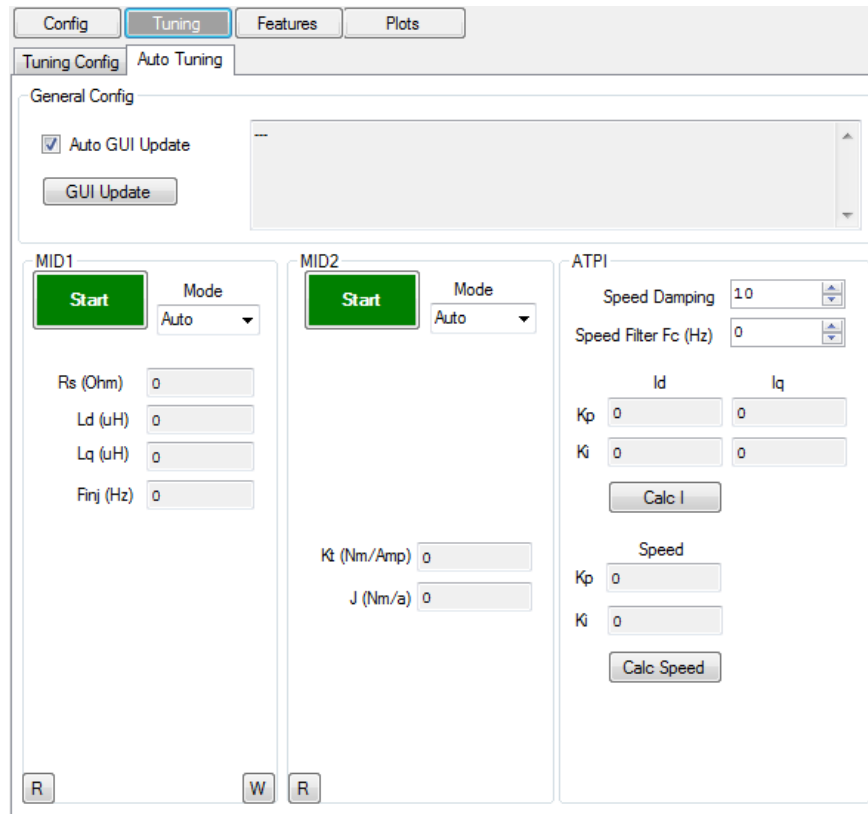


Figure 1: AutoTune Tab

3.2.1 General Configuration

Controls:

- **GUI Update:** Manually updates the PI gain and motor parameter values in the Tuning and Config panels
- **Auto GUI Update:** If selected, the PI gain and motor parameter values in the Tuning and Config panels are automatically updated every time they are calculated by the ATPi module.

3.2.2 MID1

Controls:

- **Mode:** Select between Auto, Semi Auto, and Manual modes for MID1 sequence. At this time only Auto mode is supported. Semi Auto and Manual modes are for internal debugging.
- **Start:** Start the MID1 sequence.
- **I1:** (*Semi-Auto mode only*) First current level applied to motor to calibrate phase resistance calculation. This is also DC offset applied while injecting AC signal. Units are in percentage of max current setting. Default value = 0.25.
- **I2:** (*Semi-Auto mode only*) Second current level applied to motor to calibrate phase resistance calculation. Units are in percentage of max current setting. Default value = 0.50.
- **Iac:** (*Semi-Auto mode only*) Amplitude of AC injected current for motor parameter calculation. Units are in percentage of max current setting. Default value = 0.125.
- **Inject Freq Div:** (*Semi-Auto mode only*) Defined frequency divider to generate the injection frequency. Units are $T_{\text{injection}} / T_{\text{control}}$. Default value = 10.

Displays:

- **Rs:** Measured phase resistance
- **Ld:** Measured phase inductance in the D-axis
- **Lq:** Measured phase inductance in the Q-axis
- **Finj:** Injection frequency in Hz

3.2.3 MID2

Controls:

- **Mode:** Select between Full Auto, Semi-Auto, and Manual modes for MID2 sequence. At this time only Auto and Semi-Auto modes is supported. Manual mode is for internal debugging.
- **Start:** Start the MID2 sequence.

Displays:

- **Ke:** Calculated motor BEMF constant
- **J:** Calculated motor load inertia

3.2.4 ATPI

Controls:

- **Speed Damping:** Damping factor for speed PI controller response
- **Speed Filter Fc:** Low-Pass-Filter cutoff frequency applied to speed estimation
- **Calc I:** Calculates the Current PI gain values based on the motor parameters entered in the GUI
- **Calc Speed:** Calculates the Speed PI gain values based on the motor parameters entered in the GUI

Displays:

- **Id Kp:** Calculated proportional gain for flux current (id) PI controller
- **Id Ki:** Calculated integral gain for flux current (id) PI controller
- **Iq Kp:** Calculated proportional gain for torque current (iq) PI controller



- **Iq Ki:** Calculated integral gain for torque current (iq) PI controller
- **Speed Kp:** Calculated proportional gain for speed PI controller
- **Speed Ki:** Calculated integral gain for speed PI controller



3.3 Auto-Tuning Procedure

3.3.1 Initial Steps

Listed below are the steps to prepare for Auto-Tuning:

- 1) Enter the following motor parameters in the “Motor Config” section of the Config panel of the GUI
 - a. Pole Pairs
Refer to FOC Manual Tuning Guide for instructions on how to determine the number of pole pairs if it is not available.
 - b. Optional: If values for R_s , L_d , & L_q are available these can also be entered and the MID1 sequence can be skipped.
 - c. Optional: If values for K_e or J are available, these can also be entered and the MID2 sequence can be skipped.
- 2) Enter appropriate values for the remaining items in the “App Config”, “Control Config” and “Hardware Config” sections of the “Config” panel. The FOC Tuning Guide can be used as reference to determine appropriate values.
- 3) Enter appropriate values for the following tuning parameters in the ATP1 section of the Auto-Tuning tab:
 - a. Fc-speed
Enter the desired speed filter cutoff frequency
 - b. Speed Damping
Enter the desired speed controller response damping factor according to the following guidelines:
 - i. A good standard value to start is 10 which should result in the fastest speed PI controller response without significant overshoot.
 - ii. Values below 10 result in faster response but increasing overshoot. A value of 1 is not achievable as the closer the damping factor gets to 1 the more unstable the system response becomes.
 - iii. Values larger than 10 become more underdamped. Larger values can be used if a slow response to changes in target is desired. Values above 40 may not fully converge to the target speed.
- 4) Enter an appropriate value for I_{qrefCL} in the Operation section of the GUI. A good starting point may be 5% of the maximum motor phase current in units of Amps.
- 5) Click on “Write All Parameters” to ensure that all GUI values are transferred to the firmware.

3.3.2 Running Auto-Tuning

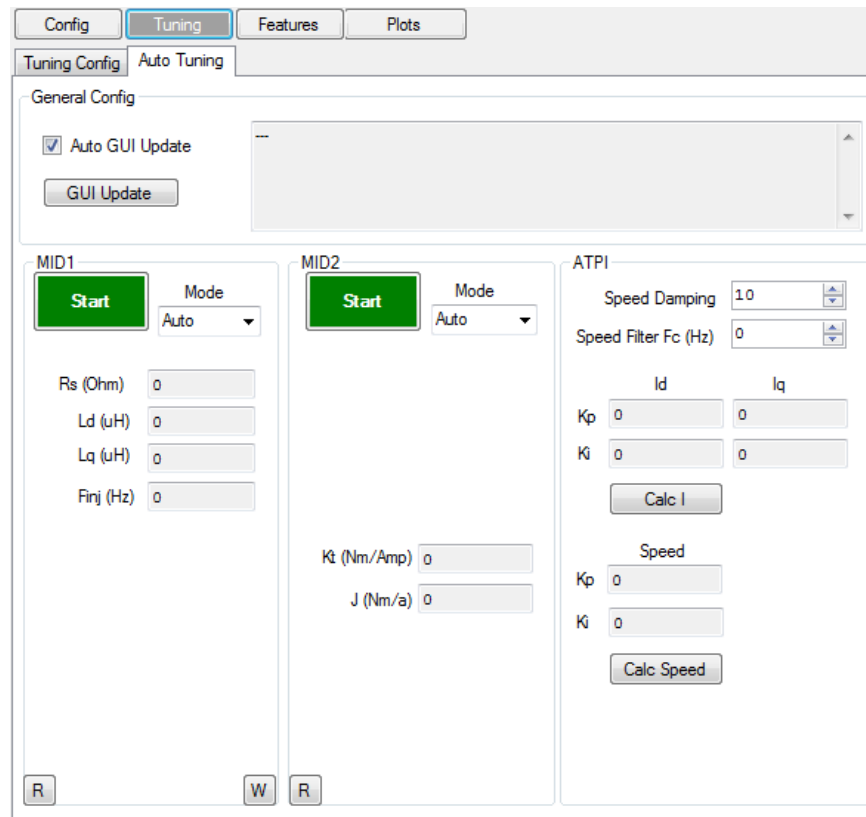


Figure 2: AutoTune Tab

Once the initial steps listed above are completed, the auto-tuning procedure can be started:

MID1

- 1) Ensure that “Auto” mode is selected in the Run Mode drop-down menu.
- 2) Click “Start” in the MID1 section to start the sequence.
- 3) The Auto-Tuning status window displays when the sequence is complete and the new measured values for R, Ld, and Lq are updated.

Note: During MID1, the current level targets are internally set by default and the injection frequency is internally determined. If for any reason it is necessary to override these values (Motor experiences an Over-Current event during MID1 for example), Semi-Auto mode can be selected and the process restarted.

Current ATPI

- 4) Click the “Calc I” button in the ATPI section to calculate the current controller gain values based on the entered parameters and on the values of Rs, Ld, and Lq that were measured during the MID1 sequence.
- 5) The updated values for the current PI controller gains are displayed in the appropriate boxes.

Startup and MPOS Tuning



- 6) Tune startup and MPOS according to the MPOS method selected in the project. At the end of this step the motor should be able to run in torque mode.

MID2

- 7) Ensure that "Auto" mode is selected in the Run Mode drop-down menu.
- 8) Motor must be enabled and running in torque mode.
- 9) Ensure that the motor speed is as close as possible to the Open-Closed Loop switchover speed. Adjust "IqrefCL" in the Operation section to achieve this.
- 10) Click "Start" in the MID2 section to start the sequence.
- 11) The Auto-Tuning status window displays when the sequence is complete and the new measured values for K_e , and J are updated.

Speed ATP

- 12) Click the "Calc Speed" button in the ATP section to calculate the speed controller gain values based on the entered parameters and on the values of K_e and J that were measured during the MID2 sequence.
- 13) The updated values for the speed PI controller gains are displayed in the appropriate boxes.

3.4 Warnings and Faults

The possible warnings and faults that can be reported by the ATPi are listed below:

- 1) **`WARNING_ATUNE_MID1_PHASECHK`**
MID1 has detected a moderate phase current imbalance between the 3 motor phases. This could be caused by an issue with the motor wires, a defect in the motor, or an issue with the motor driver. This warning only alerts the user of the imbalance but it does not stop the MID1 sequence from running. The resulting motor parameter values may have some error due to this issue.
- 2) **`FAULT_ATUNE_MID1_PHASECHK`**
MID1 has detected a large phase current imbalance between the 3 motor phases. This could be caused by an issue with the motor wires, a defect in the motor, or an issue with the motor driver. This fault stops the MID1 sequence from running since this issue prevents the motor ID from running properly.
- 3) **`WARNING_ATUNE_MID1_MOTIONDET`**
MID1 tries to wait until no motion is detected from the motor before trying to measure the inductance and could. This warning is issued whenever the MID1 sequence is unable to detect that the motor is stopped before proceeding with the measurement and could result in some error in the measured motor parameters.
- 4) **`WARNING_ATUNE_MID2_CTRLMODE`**
If the MID2 sequence has been started and the motor is either not enabled or not in Torque mode, this fault state will be entered and notify the user that the correct mode must be entered before proceeding.
- 5) **`WARNING_ATUNE_MID2_NOISECHK`**
After clicking "Calculate" in the MID2 section, if the values obtained experienced too much noise, then a larger value for Istep may be required. If this happens, the MID2 module will reset the speed to the starting point automatically and allow the user to retry:
 - a) Increment the value of Istep by 5% of the maximum phase current
 - b) Continue with steps #12 and #13 from the ATPi procedure above
- 6) **`STATUS_ATUNE_MID2_FAILED`**
After the retry limit for **`WARNING_ATUNE_MID2_NOISECHK`** is reached, then the MID2 module will enter a fault state and stop retrying. At this point the user can exit this fault state by clicking the Start button. The retry limit is configurable in `autotune_pi_funcs.h`
- 7) **`WARNING_ATUNE_ATPI_CALCNEEDED_I`**
When the user changes motor and system parameters that affect the current PI gain values, this warning alerts the user that the gains need to be recalculated.
- 8) **`WARNING_ATUNE_ATPI_CALCNEEDED_SPD`**
When the user changes motor and system parameters that affect the speed PI gain values, this warning alerts the user that the gains need to be recalculated.



3.5 Saving the Calculated Values

Once motor parameter values are measured (R_s , L_d , L_q , K_e , and J) and appropriate values are obtained for PI controller gains (K_{pd} , K_{id} , K_{pq} , K_{iq} , K_{pv} , and K_{iv}), they can be saved into an XML file for future use without having to repeat the ATP procedure. After completing the ATP procedure please follow the following steps:

- 1) If the Auto GUI update box is checked in the General Configuration section of the Autotune tab, then the values get automatically updated into the corresponding fields in the Motor Config, Current PI Tuning and Speed PI tuning sections as the values get calculated. Since this step is automatic then Step #2 can be skipped.
- 2) Click "Update GUI" button to update the calculated values from the Autotune tab into the corresponding fields in the Motor Config, Current PI Tuning and Speed PI tuning sections.
- 3) Save the values into an XML file by going to -> File -> Save All Settings

Note: It is recommended to check the Auto GUI update box in order to keep all the motor configuration and controller gains synchronized.

3.6 Firmware Parameters

Auto-Tuning uses the firmware specified configuration parameters below. They should not need to change unless issues are encountered during the auto-tuning process. These parameters are defined in mid1_funcs.h and mid2_funcs.h:

MID1

MID1_CURRENT_LIMIT1_PERCENT_Q14

Default value for V1_Align parameter in Full Auto mode. Reduce value if motor current is too large during parameter measurement.

MID1_CURRENT_LIMIT2_PERCENT_Q14

Default value for V2_Align parameter in Full Auto mode. Reduce value if motor current is too large during parameter measurement.

MID1_MOTIONDET_DELTA_THRESHOLD_PCT_Q14

Percentage of DC offset current value used to detect motion before motor parameter ID starts. Decrease value to increase sensitivity to motion. Increase value if motion is falsely detected.

MID1_MOTIONDET_DELAY_LIMIT_SEC_Q16

Defines sample time for motion detection. This sample time must be long enough to detect motion for the slowest moving motors.

MID1_MOTIONDET_TO_LIMIT_SEC_Q16

Motion detection timeout. If motion detection times out, MID1 sequence will continue and will alert user that the motor was possibly experiencing motion during the measurement.

MID1_FILTER_SHIFT_Q0

Defines filter gain for all MID1 metrics. The alpha gain is defined as $1/(2^{\text{shift}})$.

MID2

MID2_ANOISE_DELAY_CYCLES

Length of delay between the moment the Istep control is triggered and the moment that the actual current step occurs. System values used by the ATP module are sampled during this time. The delay is in units of systick cycles. If amax noise floor measurement is inconsistent, this value should be increased.

MID2_ANOISE_THRESHOLD_MULTP

Defines the threshold used for the amax noise check. After the noise floor is collected, the threshold is set to be the noise floor multiplied by this multiplier.

MID2_ANOISE_FAULT_RETRIES

Retry limit when the amax noise check fails before the sequence enters a fault state.

MID2_FILTER_FCDIV_Q16

Divider for ATP filter cutoff frequency. $F_{c_ATP} = f_{\text{unitygain}} / FCDIV$. If amax, iq, speed, or Vphase measurements are inconsistent, this value should be increased.

MID2_FILTER_FC_ACCDET_Q16

Filter cutoff frequency applied to acceleration calculation

MID2_ACCDET_THRESH_Q22

Acceleration value threshold when detecting if motor speed has settled



MID2_SPEED_SETTLED_MAJFILT_LIMIT

Count in systick clocks for blanking the motor speed settling detection

MID2_IQREF_INC_PCT_Q16

Increment for applied to iqref when stepping the current and for current step retries

3.7 Backward Compatibility

The values for the PI controller gains used in this version of firmware and GUI are different from the values that were used in previous versions. If the user wishes to use the manually tuned values from previous versions or to compare those previously used values with the ATPF values, the user must convert the old values to the new format by following the procedure outlined below:

- 1) Obtain the following system values:
 - Bus voltage (Vbus) : Typical system application voltage
 - Modulator Gain (Mod_gain) : Set to 1.15 for SVM modulation)
 - Current scale factor (I_scale) : Read from GUI v4.0 or from debug export in older versions
 - Speed scaling shifts (Speed_shifts) : Read from debug export
- 2) Calculate the following scale factors:
 - Voltage scale factor ($V_scale = Vbus / 2 * Mod_gain$)
 - Speed scale factor ($Spd_scale = 2^{Speed_shifts}$)
- 3) Convert Current PI Gains:
 - $Kpi_new = Kpi_old * (V_scale / I_scale)$
 - $Kii_new = Kii_old * (Kpi_new / Kpi_old)$
- 4) Convert Speed PI Gains:
 - $Kpv_new = Kpv_old * (I_scale / Spd_scale)$
 - $Kiv_new = Kiv_old * (Kpv_new / Kpv_old)$



CONTACT INFORMATION

For the latest specifications, additional product information, worldwide sales and distribution locations:

Web: www.qorvo.com

Tel: 1-844-890-8163

Email: customer.support@qorvo.com

IMPORTANT NOTICE

The information contained herein is believed to be reliable; however, Qorvo makes no warranties regarding the information contained herein and assumes no responsibility or liability whatsoever for the use of the information contained herein. All information contained herein is subject to change without notice. Customers should obtain and verify the latest relevant information before placing orders for Qorvo products. The information contained herein or any use of such information does not grant, explicitly or implicitly, to any party any patent rights, licenses, or any other intellectual property rights, whether with regard to such information itself or anything described by such information. **THIS INFORMATION DOES NOT CONSTITUTE A WARRANTY WITH RESPECT TO THE PRODUCTS DESCRIBED HEREIN, AND QORVO HEREBY DISCLAIMS ANY AND ALL WARRANTIES WITH RESPECT TO SUCH PRODUCTS WHETHER EXPRESS OR IMPLIED BY LAW, COURSE OF DEALING, COURSE OF PERFORMANCE, USAGE OF TRADE OR OTHERWISE, INCLUDING THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE.**

Without limiting the generality of the foregoing, Qorvo products are not warranted or authorized for use as critical components in medical, life-saving, or life-sustaining applications, or other applications where a failure would reasonably be expected to cause severe personal injury or death.

Copyright 2019 © Qorvo, Inc. | Qorvo is a registered trademark of Qorvo, Inc.